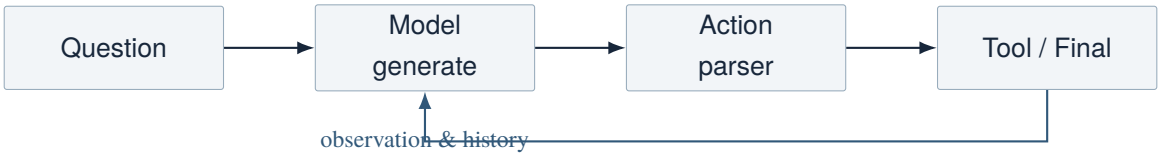
从零实现一个 ReAct Agent

Building a ReAct Agent from Scratch

动作协议 • 工具调用 • Agent 循环 • 多跳问答评测



Version	4.0 (LaTeX edition)
Date	23 October 2026
Recommended stack	Python 3.11+ / uv / pytest
Models	DeepSeek API or local Ollama (Qwen2:7B)
Benchmark	HotpotQA, distractor (closed-candidate setting)

摘要

本作业要求你**独立实现模型之外的 Agent 推断系统**：动作解析器、工具注册与执行、检索与查阅、消息管理、带预算的 ReAct 循环、模型后端、轨迹记录，以及基于 HotpotQA 的离线评测。每一节先解释必要知识，再明确输入输出、异常约定、实现步骤与验收测试；一个模块验收完成后，才进入下一个模块。你**不需要训练语言模型，不需要 GPU，也不需要 Agent 框架**。与 Stanford CS336 Assignment 1 相同，本讲义给出详细功能契约而不提供实现答案。本文件是独立编写的学习作业，并非 Stanford 官方课程材料。

目录

1 Assignment overview: 最终要完成什么 **2**

1.1 成品的行为与适用范围 . . . 2

1.2 你将实现 / 运行 / 不允许使用 . . . 2

1.3 推荐仓库结构：测试和实现分离 . . . 2

1.4 评分与逐级里程碑 . . . 4

2 Foundations: 模型、工具与环境（6 分） **4**

3 Action protocol: 先让程序读懂模型（12 分） **5**

3.1 固定的两种输出格式 . . . 5

3.2 Public interface 与建议拆解 . . . 6

4 Retrieval: 从三篇文档开始（12 分） **7**

4.1 固定 Toy Environment 与句子编号 . . . 7

4.2 最小检索算法：词集合重叠 . . . 7

5 Tool executor: 受控地执行动作（10 分） **9**

6 Messages, prompting, and the ReAct loop（20 分） **10**

6.1 消息格式与可见历史 . . . 10

6.2 状态迁移表：尤其关注边界 . . . 11

6.3 用脚本模型隔离循环错误 . . . 11

7 Provider adapters: DeepSeek 与 Ollama（10 分） **12**

8 Trace, replay, and experimental integrity（6 分） **14**

9 HotpotQA: 将 Toy Environment 扩展到真实数据（6 分） **15**

9.1 禁止金标准泄漏 . . . 15

10 Metrics and batch evaluation（12 分） 16

10.1 固定切分：先 smoke，再 dev，最后 eval 17

10.2 Batch 运行与官方预测格式 17

11 Controls, ablation, and error analysis（6 分） 19

12 Submission checklist 与常见故障定位 19

12.1 最终提交 19

12.2 逐项自检 20

12.3 遇到错误时先看什么 20

13 Optional extensions（不计基础 100 分） 20

A Interface reference: 全部公共功能速查 21

B Test matrix: 先写哪些测试，不用写任何答案代码 22

C Reference materials 22

1 Assignment overview: 最终要完成什么

1.1 成品的行为与适用范围

给定一个自然语言问题，模型每次生成一个机器可读的动作：调用 `search`、调用 `lookup` 或提交 `final`。你的程序解析并校验动作，在权限与预算内调用工具，将观察（observation）追加到消息历史，再请求模型做下一步决策。终止时输出答案、引用句子、状态和完整执行记录。

Knowledge: ReAct 在这里究竟是什么?

ReAct 的核心是交替发生的 *reason/act/observe*：基于当前可见信息形成下一步决策、作用于外部环境、读取新信息并重新决定。经典论文中的轨迹可以展示文字化 Thought；本作业为兼容不同后端使用**结构化 JSON 动作 + 可见观察**，允许模型额外给出简短的决策备注，但**不要求输出隐藏思维链**。因此你实现的是 ReAct 思想的轻量工具使用 Agent，而不是复制原论文所有 prompting 模板。它是推断系统，而非强化学习训练。

1.2 你将实现 / 运行 / 不允许使用

What you implement. (i) 严格 JSON action parser；(ii) deterministic retrieval；(iii) safe tool executor；(iv) prompt、history 与 Agent loop；(v) 两种模型连接入口；(vi) trace/JSONL；(vii) HotpotQA loader 与 per-question tools；(viii) EM/F1、批量运行和实验比较。

What you run. 首先零网络通过单元测试；然后在三个玩具文档上运行脚本假模型；之后用 DeepSeek 或 Ollama 进行一次真实端到端问答；最后在 HotpotQA 固定子集上统计真实指标。所有真实测分由你执行，不把文档示意数据写成实验结果。

What you may use. Python 标准库、`pytest`、`httpx`、`openai` SDK（仅作 *LLM transport*）、`python-dotenv`、`uv`、HotpotQA 官方评分脚本。**不可**用 LangChain、LangGraph、CrewAI、AutoGen 等代替 parser、executor、history、loop 或评测；不可用 `eval/exec` 执行模型文字；不可直接复制第三方 ReAct Agent 实现。

AI 使用建议. 概念解释、查阅库 API、理解报错可以使用 AI。为保持从零实现的训练价值，核心组件应自己动手写，尽量不要让 AI 自动补完整个函数；任何外部帮助可在报告中声明。

1.3 推荐仓库结构：测试和实现分离

```
react-from-scratch/
  pyproject.toml                # you create / edit
  .env.example                  # no real secrets
  README.md
  adapters.py                   # thin public API forwarding only
  react_agent/
    __init__.py
    protocol.py                 # you write
    retrieval.py                # you write
    tools.py                    # you write
    prompting.py                # you write
    agent.py                    # you write
```

```
    providers.py                # you write
    tracing.py                  # you write
    dataset.py                  # you write
    metrics.py                  # you write
    evaluation.py               # you write
  tests/                        # create tests; NEVER modify to hide bugs
    conftest.py
    test_protocol.py
    test_retrieval.py
    test_tools.py
    test_loop.py
    test_metrics.py
    test_dataset.py
  scripts/
    check_backend.py
    make_splits.py
    evaluate.py
  data/                         # downloaded dataset; not in code zip
  reports/
    writeup.md
    splits/
    runs/
```

这些是**建议文件名**，不是随本 PDF 附送的源码。本作业只交付讲义和接口规范；你需要自己创建代码与测试文件。`adapters.py` 应只负责将固定公共接口转发至你编写的实现，并不得容纳核心业务逻辑。使用单独的测试文件逐步自测，无需先造出完整项目。

1.4 评分与逐级里程碑

阶段	从无到有的产物	分值	进入下一阶段前的证据
M0	概念与工程环境	6	能说清模型/工具/控制器
M1	JSON 动作协议	12	解析器正反例通过
M2	候选文档检索	12	search + lookup 稳定
M3	安全工具执行	10	未知/错误工具可控
M4	消息与 ReAct 循环	20	toy 问题完整跑通
M5	DeepSeek / Ollama 后端	10	一次真实端到端调用
M6	Trace 记录与续跑	6	能复盘每个决策与工具
M7	HotpotQA 数据隔离	6	证据编号与 gold 隔离
M8	指标与批量评测	12	可对固定样本重算分数
M9	对照、消融、分析	6	可解释的真实结果
	Total	**100**	

Milestone: 从哪里开始（不需要 API Key）

先运行 `uvinit` 创建项目，再加入 `pytest`。只完成 §3 的两个合法 JSON 解析例子和五个非法例子。在第一次运行成功之前，不需要下载 HotpotQA，不需要本地模型，也不需要网络搜索。

2 Foundations: 模型、工具与环境（6 分）

Knowledge: LLM 不是能够自行执行 Python 的进程

Model 只是映射 $M : \mathcal{H} \to \mathcal{Y}$，输入为带角色的文本消息，输出为文本。**Tool** 是可执行的受控函数 $T(a)$，不应由模型直接任意调用。**Controller** 在二者之间维护状态，负责验证动作、工具权限、预算、异常、历史与终止条件。**Environment** 是本次问题可以被检索的一组候选资料。

令 h_t 为第 t 轮模型可见的消息历史，a_t 为解码并验证后的动作，o_t 为工具执行的 observation：

$$a_t \sim \pi_\theta(\cdot \mid h_t), \qquad o_t = T(a_t), \qquad h_{t+1} = h_t \oplus \text{assistant}(a_t) \oplus \text{user}(o_t).$$

其中 $\oplus$ 表示按顺序追加消息。θ 是**已训练模型参数**，本作业不更新它。h_t 是文本历史而非神经网络隐藏状态；将日志写入磁盘并不会自动让模型看到内容。

Problem (foundations, 6 points)

在 `reports/writeup.md` 中回答下列问题，每问 2 分，建议每问 100–180 字。

1. **Component boundaries.** 为什么在 system prompt 中写「最多调用六次工具」仍无法强制模型停止？谁必须检查预算？
2. **Trajectory.** h_t 与强化学习的 trajectory 有什么类似处？为什么这里不需要 reward model、policy gradient 或反向传播？
3. **Threat model.** 网页、检索结果或工具异常消息出现「忽略上面的规则」时，为什么应当把它视为数据而不是指令？

Deliverable. 三段短答，并画出至少一次 tool call 后接 final 的消息流（可手绘后写入报告）。

3 Action protocol: 先让程序读懂模型（12 分）

Knowledge: 语法正确不等于动作可执行

原始模型输出不可信：可能是自然语言、带 Markdown 围栏的 JSON、顶层列表、缺少必填字段，或者请求不存在的工具。应分**语法**（字符串能否完整解析为 JSON）、**结构**（JSON 是否满足动作契约）、**执行**（工具是否登记、参数与预算是否允许）三级验证。本节只做前两级；下一节才执行动作。

3.1 固定的两种输出格式

每次 `model.generate(messages)` 必须返回**恰好一个 JSON 对象**，可以前后留空白，不得有代码围栏、说明段落、第二个对象。模型可以省略 `note`；它只是简短可见的动作理由，既不参与评分，也不要求写完整思维链；如果出现，则必须是字符串。

```
{"type":"tool_call", "tool":"search",
 "args":{"query":"Ada Lovelace father", "k":3},
 "note":"Find the first relevant article"}

{"type":"final", "answer":"London",
 "evidence":[["Ada Lovelace",1],["Lord Byron",1]]}
```

字段	所属动作	精确定义
`type`	全部	必须为字符串：`tool_call` 或 `final`
`tool`	tool call	非空字符串，前后空白去除后不能为空
`args`	tool call	JSON object；空 object 合法，不允许数组
`answer`	final	字符串；允许空，评测仍按空答案计分
`evidence`	final	可省略，缺省为 `[]`；否则为句子引用列表
`note`	任意	可选的简短字符串；不要要求长篇隐式推理

证据的每个元素恰为 `[title,sent_id]`，其中 `title` 是非空字符串，`sent_id` 是**从零开始的非负整数**。Python 中 `bool` 是 `int` 的子类，`True` 或 `False` 仍必须拒绝。解析器**不**判断 title 是否存在，也**不**判断 tool 是否登记：这些依赖运行时环境。

3.2 Public interface 与建议拆解

`parse_action(text:str)->dict`

Purpose: 将整段 assistant 文字转换为可信的、具有固定结构的 action 字典。

Contract: 输入必须为字符串；成功时至少返回 `type` 及对应必填字段；省略 `evidence` 时补 `[]`；允许忽略无关额外字段。

Failure/edge: 遇到任何不合法 JSON/结构，统一抛出 `ValueError`；不得尝试执行工具。

先用 `json.loads`，再判断顶层是否字典，再按 `type` 分支逐项验证；不要一开始写复杂 JSON Schema 框架。

Problem (protocol-valid, 5 points)

实现 `parse_action` 的合法路径。最小增量顺序：**(a)** final + answer；**(b)** final + evidence；**(c)** tool call + tool/args；**(d)** 保留可选 note。每增加一种路径就立即补一条测试。

```
input:  '{"type":"final","answer":"Paris"}'
output: {"type":"final","answer":"Paris","evidence":[]}
```

Deliverable. 一个只包含解析逻辑的模块与四类通过的测试；不得将原始字符串直接作为 action 返回。

Problem (protocol-invalid, 5 points)

拒绝：空文本、`notjson`、`[]`、`null`、未知 type、tool 为空、args 为数组、answer 为数字、evidence 为 object、负数或布尔编号、JSON 后多余说明、连续两个 JSON 对象。错误消息尽量提到字段名。**Deliverable.** 每种至少一条负向测试，全部要求抛 `ValueError`。**Check.** 解析失败绝不能调用工具或网络。

Problem (protocol-boundary, 2 points)

为什么合法 JSON `"type":"tool_call","tool":"missing","args":{}` 应通过解析，却会在执行层失败？在报告中用 4–6 句话解释。

Milestone: M1

有了可信 action，但尚无任何工具。只要 parser 所有测试通过，就能继续；不要因为缺少模型而停住。

4 Retrieval: 从三篇文档开始（12 分）

Knowledge: 把“找到文章”与“阅读文章”拆成两个动作

`search(query,k)` 是索引/标题定位；`lookup(title)` 才返回文章句子。这样 Agent 可以执行两跳：先在 Ada 文档看到 Lord Byron，再去查 Lord Byron 的出生地。检索器应先用简单、确定性的基线建立程序正确性，而非立刻引入 embedding、向量数据库或互联网搜索。

4.1 固定 Toy Environment 与句子编号

```
toy_context = [
 ["Ada Lovelace", [
  "Ada Lovelace worked on the Analytical Engine.",
  "Ada's father was Lord Byron."]],
 ["Lord Byron", [
  "Lord Byron was a British poet.",
  "He was born in London."]],
 ["Other Document", [
  "A paragraph about plants and gardening."]]
]
```

`context` 是 `list[[title,sentences]]`。每个原始句子的列表下标就是正式证据编号，因此 Ada 的父亲关系是 `["AdaLovelace",1]`，Byron 的出生地是 `["LordByron",1]`。**禁止**重分句、排序、去重或重新编号；否则 benchmark 的证据评分会错位。

4.2 最小检索算法：词集合重叠

Knowledge: 从可复现算法学检索，而不是先追求高分

把 query 与每篇文档的“标题 + 全部句子”各自转小写，用正则 `[a-z0-9]+` 提取 token 并去重。令 Q 为 query 词集合，D_i 为第 i 篇的词集合，则

$$s(q, d_i) = |Q \cap D_i|.$$

仅保留 $s > 0$ 的文档，按分数**降序**、标题**字典序升序**排序，然后取前 k 个。此算法不是 BM25，也不能利用 gold answer 或 supporting facts 影响排序。

`tokenize_text(text:str)->set[str]`

Purpose: 为词集合检索提供统一分词规则。

Contract: 英文数字以正则 `[a-z0-9]+` 切分，大小写不敏感，返回去重 token 集合。

Failure/edge: 非字符串输入抛 `ValueError`；空字符串返回空集合。

`run_search(context:list,query:str,k:int=3)->list[str]`

Purpose: 只返回按相关度排序的文章标题。

Contract: 词集合交集分数；按分数降序、原标题升序；`k=0` 返回 `[]`；无命中返回 `[]`。

Failure/edge: 非字符串 query、负数或布尔/non-int k 抛 `ValueError`；不能返回完整正文或 gold。

`run_lookup(context:list,title:str)->dict`

Purpose: 依据准确标题阅读文档；保留原文和位置。

Contract: 精确大小写匹配，返回 `"title":str,"sentences":list[str]`，句子次序不变。

Failure/edge: 未知标题或非字符串 title 抛 `ValueError`；不能悄悄选“最相似标题”。

Problem (retrieval-basic, 8 points)

1. 只实现 `tokenize_text`，检查 `"Britishpoet"` 得到两个 token。
2. 实现计算单文档交集分数的小函数（内部名称自定）；在 Toy Context 纸笔核对分数。
3. 实现所有文档排序与 top-k；用 `run_search(toy_context,"AnalyticalEngine",2)` 验收 `["AdaLovelace"]`。
4. 实现 `run_lookup(toy_context,"AdaLovelace")`；确认返回两句原文。

Deliverable. 对合法查询、空查询、无命中、同分排序、k=0/负数/布尔值、未知 title 分别写测试。

Problem (retrieval-proof, 4 points)

在报告中分别解释：为何使用集合而不是词频计数；`search` 与 `lookup` 分开可以观测哪两种失败；为何标准答案不能加入检索分数。附一个自行构造的同分排序例子。

Milestone: M2

不调用 LLM, 也可以在 Python REPL 中手动完成 `search("Ada...")`、`lookup("AdaLovelace")`、`lookup("LordByron")`。这一阶段产物是纯检索函数，不是 Agent。

5 Tool executor: 受控地执行动作（10 分）

Knowledge: “模型请求执行”不等于“程序授权执行”

工具注册表是 $R = \{\text{name} \mapsto \text{callable}\}$。控制器仅能从显式注册表选工具，不能把模型生成的名字送给 `globals()`、反射模块任意属性或 shell。将动作参数作为关键字参数绑定给允许的函数。工具异常必须转换为可供模型阅读的失败观察，不能让整个循环意外崩溃。

`execute_tool(action:dict,tools:dict,max_observation_chars:int=8000)->dict`

Purpose: 验证执行条件、调用已登记的工具、统一观察格式。

Contract: 成功返回 `"ok":true,"observation":"..."`；字符串原样保留，字典/列表确定性 JSON 序列化，其他值需清楚声明策略。

Failure/edge: 未知工具、参数不匹配、工具异常及超长输出返回 `ok=false` 与以 `ERROR:` 开头的字符串；不能触发其他函数。

`make_tools(context:list)->dict`

Purpose: 为当前一道题创建工具注册表。

Contract: 返回至少 `search`、`lookup` 两个 callable；通过闭包或对象绑定 `context`，模型可见参数仅为 `query,k` 或 `title`。

Failure/edge: 每道题的 context 必须隔离，工具不能读取上一题资料。

观察超长时，原则是**不能将无限文本放入历史**：在 8000 字符以内返回带截断标志的可见 preview，并且 `ok=false`。计数上，即使工具不存在或参数错误，仍算一次工具调用尝试。捕获 `Exception`，不要捕获 `KeyboardInterrupt` 等退出信号。

Problem (tool-registry, 6 points)

先使用 `"echo":echo` 测试注入工具，而**不是**直接依赖 search/lookup。顺序：成功 echo；未知名称；少参数；多参数；函数主动抛异常；返回字典/列表；长字符串截断。**Deliverable.** 一个工具适配层及至少七个测试。

Problem (context-binding, 4 points)

为 A、B 两个互不相交的 toy context 分别调用 `make_tools`。证明 A 的 `lookup` 无法读出仅存在于 B 的标题。**Deliverable.** 两组 registry 和隔离测试；说明闭包绑定为何比让模型传 `context` 更安全。

Milestone: M3

此时你可以手工构造合法 action 字典，经过 `execute_tool` 获取结构化观察；下节只需给它加上模型决策与循环。

6 Messages, prompting, and the ReAct loop（20 分）

Knowledge: Agent 是一个带预算的有限状态机

状态至少包含 h_t（messages）、c_m（已进行的逻辑模型调用次数）、c_t（已尝试的工具次数）、trace。每次模型调用使 $c_m \leftarrow c_m + 1$；当模型给出 tool call 且仍有额度时，$c_t \leftarrow c_t + 1$。工具失败也计数。final 立刻成功；非法输出/模型异常返回 error；超出模型次数返回 max_steps；无工具额度而模型还要求工具时返回 tool_limit。两种预算不能混在一起。

6.1 消息格式与可见历史

```
initial messages:
[
 {"role":"system", "content":"<protocol + tool instructions>"},
 {"role":"user", "content":"Where was Ada Lovelace's father born?"}
]

after the model requests lookup:
 {"role":"assistant", "content":"{\"type\":\"tool_call\",...}"}
 {"role":"user", "content":"Observation: {\"title\":\"Ada Lovelace\",...}"}
```

注意模型每次仅能看到传入的 `messages`；工具打印结果到终端或 trace 文件不等于模型已经收到观察。本作业用 `user` role 承载带前缀的 observation，**不使用** API native tool-call role。将工具输出明显标记为外部数据，禁止它覆盖 system 规则。

`build_initial_messages(question:str,tool_descriptions:str,max_steps:int,max_tool_calls:int)->list[dict]`

Purpose: 构造初始 system + user 消息。

Contract: system 必须阐明唯一 JSON 输出协议、工具参数、证据格式、预算；user 只携带 question。

Failure/edge: 不要包含本题全部 context、gold answer 或 supporting facts；错误参数应明确拒绝。

`append_observation(messages:list[dict],raw_action:str,observation:str)->list[dict]`

Purpose: 将一次动作及其真实观察按顺序写入下轮消息。

Contract: 返回包含原有消息、assistant raw action、user observation 的新列表；观察带可区分的`Observation:`前缀。

Failure/edge: 不得遗漏原 question，不得把外部观察作为 system 指令。

`run_agent(question:str,model,tools:dict,max_steps:int=8,max_tool_calls:int=6)->dict`

Purpose: 在统一接口下执行完整决策循环。

Contract: model 具有 `generate(messages)->str`；结果至少包含 `status,answer,evidence,trace,num_llm_calls,num_tool_calls`。

Failure/edge: 非 final 时 `answer=""`、`evidence=[]`；非法预算在模型调用前抛 `ValueError`；其他失败受控停止。

6.2 状态迁移表：尤其关注边界

收到的决策	状态	必须做的事
合法 final	`final`	保存答案/证据，绝不再调用模型或工具
合法 tool call，有额度	继续	尝试执行、追加 observation、记 trace
合法 tool call，无额度	`tool_limit`	不调用工具，不增加工具尝试数
parse 错误 / generate 异常	`error`	记录原始文字与错误，不能把异常当答案
最后一步仍只产生 tool call	`max_steps`	不可“额外再请求一次模型”
`max_steps=0`	`max_steps`	初始时直接终止、零模型请求

关键反例： `max_steps=3,max_tool_calls=2` 应允许轨迹 *tool call* → *tool call* → *final* 成功！工具额度耗尽不代表必须马上停止；只有下一个动作依然要求工具时才返回 `tool_limit`。而 `max_steps=2,max_tool_calls=2` 下，第二次执行工具后不可偷偷调用第三次模型。

6.3 用脚本模型隔离循环错误

在测试里自写一个最简单的 `ScriptedModel`：构造时接收预设输出列表，`generate(messages)` 按顺序返回字符串，并保存每次收到的 messages。测试使用以下两次模型输出：

```
{"type":"tool_call","tool":"lookup",
 "args":{"title":"Ada Lovelace"}}
{"type":"final","answer":"Lord Byron",
 "evidence":[["Ada Lovelace",1]]}
```

两跳升级测试（必须做）： 当问题是 *Where was Ada Lovelaceś father born?* 时，另外构造一个三步 ScriptedModel：*lookup Ada Lovelace* → *lookup Lord Byron* → *final London*，最终 evidence 同时包括两条引用。它应该使用 3 次 generate、2 次工具调用，而不是靠模型首轮猜出 London。这个测试验证多跳推理所需的**连续观察传递**。

Problem (prompt-and-history, 4 points)

先单独测试 `build_initial_messages` 与 `append_observation`：第二次模型看到的历史仍含初始问题，并且观察中出现 `LordByron`；外部工具文字没有改变 system 指令。**Deliverable.** 可断

言的 messages 快照。

Problem (agent-happy-path, 8 points)

只用 ScriptedModel 和 Toy Context 实现最短的完整 Agent 循环。**顺序：** (1) 组 messages；(2) generate；(3) parse；(4) final 返回或执行工具；(5) observation 回填；(6) 重复。**Deliverable.** 终态 `status="final"`, `num_llm_calls=2`, `num_tool_calls=1`，第二次输入包含第一轮观察。

Problem (loop-budgets, 5 points)

对 `max_steps=0`、`max_tool_calls=0`、负数/布尔预算、*tool/tool/final*、*tool/tool/tool*、*tool at last step* 写测试。**Deliverable.** 每个终态、调用次数和工具副作用次数都与预算一致；失败工具也计入工具尝试数。

Problem (loop-failure-recovery, 3 points)

脚本化产生一次工具错误，然后模型纠正参数并提交 final；另构造无效 JSON 与生成异常。**Deliverable.** 工具错误以 observation 返回并允许自我修正，协议/模型错误则受控结束且保留 trace。基础版不强制实现自动 JSON 修复。

Milestone: M4 ——最重要的节点

这里已经是**完整的 ReAct Agent 控制系统**；只是决策由 ScriptedModel 假装提供。建议在完成本节之前**完全不要**连接付费 API。若此节点通过，接入模型只是更换 `model` 对象。

7 Provider adapters: DeepSeek 与 Ollama（10 分）

Knowledge: Transport、Action、Controller 是三种不同的接口

提供者的 SDK 返回可能包含 `choices`、`message`、`content`、usage、reasoning 等字段。`generate` 的职责只是把网络响应转为一个可供 parser 读取的 **assistant 文本字符串**；`parse_action` 才把文本变为 action；`run_agent` 决定何时执行与停止。切换后端不应修改协议/工具/循环。

`make_model(backend:str,model_id:str,base_url:str,api_key:str|None=None,timeout:float=60.0)`

Purpose: 建立一个具有 `generate(messages)->str` 方法的模型适配对象。

Contract: `backend` 支持 `deepseek` 和 `ollama`；base URL 显式传入；超时有限；逻辑输出仅为助手可见正文。

Failure/edge: 配置为空、连不上、认证错误、响应缺失或空字符串应报告受控异常；不得将错误伪装成空 final。

配置	DeepSeek	Ollama
`REACT_BACKEND`	`deepseek`	`ollama`
`REACT_BASE_URL`	`https://api.deepseek.com`	`http://localhost:11434/v1`
`REACT_MODEL`	以官方文档当前有效 ID 为准，可配置	`qwen2:7b`（或实际拉取的 ID）
`DEEPSEEK_API_KEY`	本机 `.env` 读取，不得提交	无需真实云端密钥；某些 SDK 允许占位值
`REACT_TIMEOUT`	有限超时，例如 60s	有限超时，例如 60s

本地准备：安装 Ollama，使用 `ollamapullqwen2:7b`、`ollamalist` 核对模型 ID；服务未运行时执行 `ollamaserve`。可用原生 `/api/chat` 或 OpenAI-compatible `/v1` 接口，但在报告中记录自己使用哪一个。**云端准备：**获取 DeepSeek key，查询官方 Chat Completions 文档确认当时可用 model ID，而非在代码中假定某个历史别名永远有效。

Knowledge: 重试与逻辑步数不相同

一个 `generate` 可以因瞬态 HTTP 错误进行有限重试，但它仍是一次逻辑决策步。例如 max_steps=8 且每步最多 3 次 HTTP 尝试，网络实际请求数可能达到 24。必须设置 SDK 的重试策略（或自己控制），记录 `num_http_attempts`（可以缺省为 null，但不得编造），不要对认证失败无界重试。

Problem (provider-contract, 4 points)

先伪造 SDK 成功响应，只测试正文提取；再伪造 `content=None`、缺少 choices、空白正文、超时及认证错误。**Deliverable.** 两个后端创建入口和明确的异常策略；对外仍是相同的 `generate(messages)->str`。

Problem (provider-live, 4 points)

任选**一个**后端完成：(a) 连通性请求；(b) Toy Context 上至少一次真实 Agent 运行。另一个后端需要实现配置入口但无需付费/下载后强行实测。**Deliverable.** `reports/backend.md`：后端、模型 ID、base URL、时间、真实/模拟、耗时、观察到的问题及解决方式；**绝不**记录密钥。

Problem (provider-boundaries, 2 points)

比较“模型返回不是 JSON”和“HTTP 超时”的区别：二者分别在哪层被捕获？是否应该重试？说明为什么不能以 `status=final` 伪装失败。

Milestone: M5

这一次你拥有**真正的端到端 ReAct Agent**。如果 JSON 格式遵守率不高，优先分析 prompt 与模型输出；不要为了让一题成功，就在 parser 里偷偷接受任意字符串。

8 Trace, replay, and experimental integrity（6 分）

Knowledge: 调试日志和模型上下文是不同的对象

`messages` 只包含模型可见信息；`trace` 应额外保存决策步、原始生成、解析结果、工具状态、异常、耗时、额度等，用于定位首次出错位置。最合适的批处理日志格式是 JSONL：**每行一道题的完整运行记录**。这允许中断后继续，并支持逐条查看失败样本。

```
{
 "sample_id":"toy-001", "run_id":"react-smoke-v1",
 "status":"final", "answer":"Lord Byron",
 "evidence":[["Ada Lovelace",1]],
 "num_llm_calls":2, "num_tool_calls":1,
 "elapsed_seconds":null,
 "config":{"backend":"scripted", "prompt_version":"v1"},
 "trace":[
  {"step_index":0,"raw_output":"...",
   "action":{"type":"tool_call","tool":"lookup"},
   "tool_result":{"ok":true,"observation":"..."},
   "error":null,"elapsed_seconds":null},
  {"step_index":1,"raw_output":"...",
   "action":{"type":"final","answer":"Lord Byron"},
   "tool_result":null,"error":null,"elapsed_seconds":null}
 ]
}
```

说明：上述 *null* 与省略号表示示意字段；不是一次实际运行的计时或轨迹。实际轨迹必须完整并能定位两次真实决策。

`write_jsonl_record(path,record:dict)->None`

Purpose: 将完整的单题记录附加至运行日志。

Contract: UTF-8，每条记录恰好一行有效 JSON；每条包含 sample ID、run ID、结果、轨迹、配置引用。

Failure/edge: 不要记密钥、Authorization header、隐藏推理内容；对文件末尾残缺行应显式诊断，不能静默当成功。

`load_completed_ids(path,run_id:str)->set[str]`

Purpose: 断点续跑前获得同一 run ID 下已有完整记录的样本 ID。

Contract: 仅把可解析、完整且同一 run ID 的记录视为已完成；跳过重复题。

Failure/edge: 不能因为半行 JSON 或不同 run ID 错误跳过样本；重复键应明确标记。

Problem (trace-schema, 4 points)

对成功 final、工具失败后修正、超预算三条轨迹写 JSONL。每条记录每个生成步都包含 `step_index`（从 0 开始）、`raw_output`、`action`、`tool_result`、`error`、`elapsed_seconds`；总计数字段能从步骤重算。**Deliverable.** 实际运行日志与一份说明字段语义的短文。

Problem (trace-resume, 2 points)

先写两道 toy 结果，模拟程序中断；恢复时仅运行剩余 ID，再次运行不会重复写记录。对半行损坏记录给出显式失败或修复策略。**Deliverable.** 一次成功续跑演示；在报告中说明温度为 0 也不能保证云模型绝对逐字复现。

9 HotpotQA: 将 Toy Environment 扩展到真实数据（6 分）

Knowledge: 候选集问答不是联网查整套 Wikipedia

HotpotQA dev distractor 每题提供 question、候选文档 context、answer、supporting_facts。你的 Agent 仅能在**本题候选文档集合**中 search 和 lookup：这是 closed-candidate retrieval。不是 HotpotQA fullwiki 排行榜，也不是搜索整个互联网。样本同时有 bridge/comparison 等不同推理类型，应在离线分析时按类型分组。

```
{
 "_id":"example-id",
 "question":"Where was Ada Lovelace's father born?",
 "context":[["Ada Lovelace",["...","..."]],
           ["Lord Byron",["...","..."]]],
 "answer":"London",
 "supporting_facts":[["Ada Lovelace",1],["Lord Byron",1]],
 "type":"bridge", "level":"easy"
}
```

以上是人工构造的结构例子，并非官方数据里的具体题目。

9.1 禁止金标准泄漏

字段	可访问对象
`_id`, `question`	运行入口 / 初始问题
`context`	仅该题 search/lookup 的内部闭包
`answer`, `supporting_facts`	仅离线 scorer；不得进入模型输入或工具输出
`type`, `level`	仅分组分析；不要用于推断时 oracle 选择

`load_hotpot(path)->list[dict]`

Purpose: 加载 HotpotQA dev distractor JSON 文件并校验基本结构。

Contract: 每条 `_id` 非空且全局唯一；`question` 字符串；context 为 `[title,sentences]` 列表；句子保持原顺序。

Failure/edge: 缺字段、重复 ID、同一题重复标题、非法数据类型必须明确报错；不得静默丢题。

`split_example(example:dict)->tuple[dict,dict]`

Purpose: 把运行数据和离线金标准物理分开。

Contract: 第一个字典只含 ID、question、context；第二个只含 ID、answer、supporting facts 及用于离线统计的元数据。

Failure/edge: 不允许把整条原始 example 交给 prompt 函数再“临时去掉答案”。

Problem (dataset-loader, 2 points)

下载官方 HotpotQA 仓库所列 dev distractor 数据。仅加载与验证，不调用模型；随机检查某题 1–2 篇文档的句子下标。**Deliverable.** 数据文件 SHA256、加载条数与格式校验日志（不要在报告中粘贴整份数据）。

Problem (data-isolation, 3 points)

调用 `split_example`、`make_tools`，让 Agent 输入只得到 question；在首次 tool call 前，检查消息里**没有**全部候选文档、gold answer、supporting facts。跨题测试 A/B 的工具不能互相访问。**Deliverable.** 两个可自动运行的隔离测试。

Problem (evidence-contract, 1 point)

最终 evidence 必须是原始文章标题 + 原句下标（*0-based*），不是排名或文档在候选集中的位置。若模型给出不存在的引用，不得用 gold 修正；保留预测并在 trace 标记无效引用，供离线 scorer 计分。**Deliverable.** 一道真实数据单题的轨迹。

10 Metrics and batch evaluation（12 分）

Knowledge: 答案正确与证据正确需要两套指标

答案 Exact Match (EM) 判断归一化文本是否完全相等；答案 F1 衡量词的重叠度（**重复词按出现次数**，故用 `Counter`）。若交集词数为 C，预测词数为 P，标准词数为 G:

$$\text{precision} = \frac{C}{P}, \quad \text{recall} = \frac{C}{G}, \quad F_1 = \frac{2\,\text{precision}\,\text{recall}}{\text{precision} + \text{recall}}.$$

分母零时按官方脚本定义处理；`yes/no/noanswer` 不相同的答案对 F1 必须为 0。证据 SP (supporting facts) 按 $(title, sent_id)$ 集合计算 precision/recall/F1 与 EM，重复提交同一句不加分。官方 joint 指标同时考虑答案与证据，不是二者 F1 的算术平均。

`normalize_answer(answer:str)->str`

Purpose: 实现 HotpotQA 官方英文答案归一化规则。

Contract: 依次 lowercase、删除 `string.punctuation`、删除独立冠词 a/an/the、压缩空格。

Failure/edge: 非字符串抛 `ValueError`；不要自行加入同义词或词干匹配。

`score_answer(prediction:str,gold:str)->dict`

Purpose: 计算单题 answer EM/F1/precision/recall。

Contract: 返回四个键，数值均为 0–1；重复词用 multiset，特殊答案按官方逻辑处理。

Failure/edge: 空值遵守官方 `exact_match_score` 与 `f1_score` 语义：双方空时 EM=1、F1=0。

`score_support(predicted:list,gold:list)->dict`

Purpose: 按文章标题、句子 ID 集合计算 SP 指标。

Contract: 对证据去重、顺序不敏感；返回 EM/F1/precision/recall。

Failure/edge: 空分母时 P/R/F1=0，双方空时 EM=1；标题精确匹配，不做答案文本归一化。

Problem (metric-unit-tests, 5 points)

检查 `normalize_answer("TheQUICK,BrownFox!")=="quickbrownfox"`；答案 `"AdaByron"` 对 `"AdaLovelaceByron"` 应有 $P = 1, R = 2/3, F_1 = 0.8, EM = 0$。另测 `"redredblue"` 与 `"redblueblue"` 的重复词、yes/no、空字符串、重复证据、证据顺序。**Deliverable.** 本地评分与官方 评测脚本 的一致性核验。

10.1 固定切分：先 smoke，再 dev，最后 eval

从完整官方 dev distractor 的所有唯一 `_id` 构造哈希键

$$k(id) = \text{SHA256}(\text{UTF8}(\texttt{"42:"} + id)).\text{hexdigest}().$$

按键升序排序（键相同再按 ID 字符串升序）。基础路线取前 **20/50/200** 个为 smoke/dev/eval, 切片依次为 `[0:20]`、`[20:70]`、`[70:270]`。固定切分是作业内协议, 并非官方发布的 200 题隐藏测试集。不得用进程随机化的 `hash()`。**`make_fixed_splits(ids:list[str],seed:int=42)->dict[str,list[str]]`**

Purpose: 确定性产生三组互不相交的样本 ID。

Contract: 基础设定的输出键为 `smoke,dev,eval`；各含 20、50、200 个 ID；不同进程、重复执行完全相同。

Failure/edge: 出现重复 ID、缺少 270 道或非法 ID 应失败；不可以先跑模型再挑容易题。

Problem (fixed-splits, 3 points)

生成 `reports/splits/smoke_ids.txt`（20）、`dev_ids.txt`（50）、`eval_ids.txt`（200），每行一个原始 ID。**Deliverable.** 三个文件、数据摘要和互斥测试。

10.2 Batch 运行与官方预测格式

```
{
```

```
 "answer": {"example-id": "London"},
 "sp": {"example-id": [
  ["Ada Lovelace",1],["Lord Byron",1]
 ]}
}
```

`make_predictions_json(records:list[dict],expected_ids:list[str])->dict`

Purpose: 把每题终态转成官方可读取的 `answer/sp` 两个字典。

Contract: 同一个计划 ID 必须出现在两个映射中；非 final 用空答案与空证据；键集合必须等于传入的 `expected_ids`。

Failure/edge: 重复 ID、未完成 ID、混入其他子集须显式报告；不能只导出答对的题。

`run_batch(data_path,ids_path,model_factory,method:str,config:dict,output_dir)->dict`

Purpose: 对固定子集逐题独立执行、保存预测和 trace，并给出 aggregate。

Contract: `method` 至少 `react`；逐题绑定 context；失败题计入分母；保证每题调用计数真实。

Failure/edge: 支持中断后的去重续跑；不允许将评测金标准传给模型；不能把部分题结果标成全量结果。

Problem (batch-and-official-score, 4 points)

依次执行 20 题 smoke、50 题 dev，再**冻结配置**执行 200 题 eval。每个 run 目录至少有 `config.json`、`ids.txt`、`predictions.json`、`metrics.json`、`traces.jsonl`。从官方原始数据生成仅供 scorer 使用的匹配本次 *ID* 的 *gold* 子集。官方代码需要 `ujson` 时按原环境要求单独安装，不需要修改官方评分规则。

```
# Example commands you must make executable yourself
uv run python scripts/make_splits.py --data data/hotpot_dev_distractor_v1.json
uv run python scripts/evaluate.py --method react --backend ollama \
    --ids reports/splits/smoke_ids.txt --output-dir reports/runs/react-smoke
# Official scorer's CLI shape:
python hotpot_evaluate_v1.py predictions.json subset_gold.json
```

Deliverable. Answer EM/F1、SP EM/F1、Joint EM/F1，以及平均模型/工具次数、错误率、无效动作率、平均单题耗时。所有计划样本都在分母；使用官方脚本交叉验证。

Milestone: 资源有限时的诚实报告

可以先只跑 **5 题诊断 + 20 题 smoke** 验证完整链路；若预算有限，报告实际完成的子集和样本数，但不能把它冒充 200 题正式 eval，更不能据此比较官方 leaderboard。模型未装好时只做 M0–M4/M7 的离线测试，仍然有价值。

11 Controls, ablation, and error analysis（6 分）

Knowledge: 对照实验回答机制问题，不是比谁 prompt 更长

想判断“交互式检索是否有用”，要在**同一模型、同一题目、同一评分器**上比较三种方法。调用次数不是 nuisance：ReAct 的多轮交互可能提高准确率，也可能增加延迟/成本。报告应同时展示性能、成功率和调用量。消融只改一个变量；最终 eval 不用于反复调参。

方法	严格定义	模型调用
Direct	仅问题，模型直接给 final JSON；不暴露工具和 context	1
Single Retrieval	程序先用完整 question 固定 `search(k=1)` 再 `lookup`；模型一次性读结果提交 final	1
ReAct	模型逐轮决定 search / lookup / final，max_steps=8, max_tool_calls=6	最多 8

Problem (baselines, 2 points)

固定 model ID、temperature、数据切分与检索器，先在 dev 检查三方法，再在相同 eval IDs 上比较；至少统计答案 EM/F1、SP F1、Joint F1、模型/工具次数和耗时。**Deliverable.** 一张真实表；若资源不足，清楚标注仅 dev 子集。

Problem (ablation, 2 points)

挑一个变量，例如 `max_tool_calls=6` 与 `max_tool_calls=1`；在固定 dev IDs 上比较。先写预测假设，后报告实际结果（没有提升也可以）。**Deliverable.** 一张变量控制表与两段机制讨论。

Problem (failure-cases, 2 points)

选至少 5 个失败案例（或全部实际失败再补受控故障测试）。逐个写 sample ID、首次关键偏离、原动作、观察、预测答案、离线标准答案、失败分类与可检验修正。建议类别：初始检索错误、第二跳缺失、工具参数错误、格式错误、预算耗尽、证据下标错误、截断或观察误读。**Deliverable.** 以 trace 为证据，避免空泛地写“模型能力差”。

12 Submission checklist 与常见故障定位

12.1 最终提交

1. **Code.** `react_agent/` 的全部从零实现、薄 `adapters.py`、测试、脚本、`pyproject.toml`、`README.md`、`.env.example`；不提交真实 `.env`、密钥、`.venv`、整份大数据。
2. **Writeup.** `reports/writeup.md` 与自行导出的 `writeup.pdf`；每道书面题答案、运行环境、命令、模型/配置、真实指标、消融和失败分析。

3. **Experiments.** 固定三个 ID 切分、真实运行目录、至少三条完整 trace、后端连接记录与报告中的版本信息。

12.2 逐项自检

- ☐ parser 只做解析与结构验证；任何非法动作不触发 Python 工具。
- ☐ registry 只执行注册函数；函数异常以观察返回，失败也消耗工具尝试数。
- ☐ Toy Context 中“先查 Ada、再查 Byron、输出 London”可以由 ScriptedModel 走通。
- ☐ 下一次 `generate` 的 messages 中确实包含上一轮 observation。
- ☐ tool / tool / final 在 3 个模型步、2 个工具额度下成功。
- ☐ DeepSeek 与 Ollama 都具有相同模型接口；至少一个后端有真实运行记录。
- ☐ Agent 初始消息没有全部 context、answer、supporting facts；跨题工具环境隔离。
- ☐ 原始文章标题、句子数组与从零编号都保持不变。
- ☐ 20/50/200 切分确定且互不重叠；非 final 题没有从分母消失。
- ☐ 官方 scorer 读得懂预测；模型名、日期、数据 hash 与配置可追溯。

12.3 遇到错误时先看什么

现象	首先定位
模型生成了漂亮的文字却无法执行	看 raw output 是否是唯一的 JSON object；检查 prompt
模型“看不到”已经查过的文档	检查下一轮 seen_messages，不要只查打印日志
只要第二次工具调用就异常退出	区分 max_steps 和 max_tool_calls 的计数与检查时点
本地 Ollama 的请求发到远端	检查显式 base URL 和 SDK 默认配置
同一文档证据编号总是错	检查是否重新分句、排序或把 0-based 变成 1-based
EM/F1 异常高或异常低	检查 gold 泄漏、空预测分母、官方 scorer 的输入 ID 集合
无法重现同一次结果	先排除切分/排序/版本差异，再考虑模型非确定性

13 Optional extensions（不计基础 100 分）

以下扩展需要先完整通过基础链路，且每项都应有固定对照和真实结果：**(i)** 自己写 BM25 与词集合重叠对比；**(ii)** 上下文历史压缩/去重与证据定位损失分析；**(iii)** OpenAI-compatible native tool calling 与 JSON 文本协议的格式遵守率对比；**(iv)** 模型端 self-correction（每次修正明确消耗一次逻辑步骤）；**(v)** 换成 ALFWorld/WebShop 等交互环境，重新定义动作、观察与评分协议。不要将不同 benchmark 的数字直接相互比较。

A Interface reference: 全部公共功能速查

本附录是公共函数索引；**精确签名、输入输出、异常语义全部在对应正文小节中**。下表是建议公共函数集合；未写在这里的私有辅助函数可以自由设计。为了避免“实现了函数却无法拼成系统”，请按从上到下的顺序逐步连接。

节	Function	I/O 与核心责任
2	`parse_action`	JSON 语法 + action 结构校验
3	`tokenize_text`	正则词集合
3	`run_search`	候选文章标题检索
3	`run_lookup`	返回原题句子数组
4	`execute_tool`	受控执行与观察格式化
4	`make_tools`	绑定当前题目数据源
5	`build_initial_messages`	prompt/角色/协议
5	`append_observation`	维护可见历史
5	`run_agent`	多轮控制、终态与计数
6	`make_model`	统一模型接口
7	`write_jsonl_record`	单题事件追踪落盘
7	`load_completed_ids`	断点续跑/去重
8	`load_hotpot`	数据校验与读取
8	`split_example`	运行输入与标签隔离
9	`normalize_answer`	标准答案归一化
9	`score_answer`	EM / F1 / P / R
9	`score_support`	SP EM / F1 / P / R
9	`make_fixed_splits`	复现切分
9	`make_predictions_json`	官方 scorer 预测结构
9	`run_batch`	独立批量运行、存档与统计

B Test matrix: 先写哪些测试，不用写任何答案代码

测试文件（建议）	优先写的断言	能避免的常见问题
`test_protocol.py`	valid final/tool；invalid JSON/证据字段	把无效字符串当动作
`test_retrieval.py`	query/k 边界、排序、原句不动	检索不稳定或证据错位
`test_tools.py`	echo/unknown/raise/long/context A-B	越权执行、意外崩溃
`test_loop.py`	2-step happy path，3-step 边界，状态与计数	工具看到了但模型没看到
`test_dataset.py`	样本字段、去泄漏、隔离	金标准混进 prompt
`test_metrics.py`	Counter 重复词、yes/no、empty、SP 去重	自写评分与官方偏离

C Reference materials

1. Stanford CS336 Staff. *Assignment 1 (basics): Building a Transformer LM*, version 26.0.3 (2026). https://github.com/stanford-cs336/assignment1-basics. 本作业借鉴其 problem/deliverable、从零实现、adapter、测试与低资源提醒的组织形式。

2. Yao, S. et al. *ReAct: Synergizing Reasoning and Acting in Language Models*. arXiv:2210.03629 (2022; v3, 2023). https://arxiv.org/abs/2210.03629.

3. HotpotQA official repository: https://github.com/hotpotqa/hotpot. 包含 dev distractor 数据说明、`context` / `supporting_facts` 结构和预测 JSON 格式。

4. HotpotQA official evaluator: https://github.com/hotpotqa/hotpot/blob/master/hotpot_evaluate_v1.py. 单题答案、证据及 joint 计算以此实现为最终核验依据。

5. DeepSeek API, Chat Completions: https://api-docs.deepseek.com/api/create-chat-completion/. 模型 ID 和响应参数会变化，请运行时核对。

6. Ollama OpenAI compatibility: https://docs.ollama.com/openai; model library https://ollama.com/library/qwen2. 了解本地模型及兼容接口。

7. uv documentation: https://docs.astral.sh/uv/. 用于隔离项目、管理依赖及执行测试。

Document note. 本文件不是随附的代码仓库，表中测试文件和命令均是需要你在项目中创建并实现的要求；任何示例轨迹、计时或分数都不代表真实运行结果。不要为了通过测试修改 gold labels 或缩小已经锁定的评测集。